

COGNOMS:

NOM:

Problema 2. (2.5 puntos)

Dado el siguiente código escrito en C, que compilamos para un sistema linux de 32 bits:

typedef struct	typedef struct
{	{
char c1;	char a;
unsigned short int M[3][3];	sa v[4];
char c2;	int i;
} sa;	} sb;

- a) **Dibuja** como quedarían almacenadas en memoria las estructuras **sa** y **sb**, indicando claramente los desplazamientos respecto al inicio, el tamaño de todos los campos y el tamaño de los structs.

The diagram illustrates the memory layout for two variables, `M` and `v`, with their respective offsets and sizes.

Variable M:

- Offset: `c1 <- 0`
- Size: `2B` (2 bytes)
- Content: `-` (dash)
- Offset: `c1`
- Offset: `M <- 2=1+1vacio`
- Offset: `<- 4`
- Offset: `<- 6`
- Offset: `<- 18`
- Offset: `c2 <- 20`
- Offset: `<- 22=21+1vacio`
- Size: `2B` (2 bytes)
- Content: `-` (dash)
- Offset: `c2`
- Size: `22 bytes` (alinea a 2)

Variable v:

- Offset: `a <- 0`
- Size: `2B` (2 bytes)
- Content: `-` (dash)
- Offset: `a`
- Offset: `v <- 2=1+1vacio`
- Offset: `<- 24`
- Offset: `<- 68`
- Offset: `<- 90`
- Offset: `<- 92=90+2vacios`
- Size: `4B` (4 bytes)
- Content: `i` (lowercase i)
- Offset: `<- 96`
- Size: `96 bytes` (alinea a 4)

- b) **Escribe** la secuencia de instrucciones necesaria para codificar la siguiente sentencia: $k.v[i].M[i-1][i-1] = 1$. La variable **k** es de tipo **sb** y su dirección está almacenada en el registro **%edx**. La variable **i** es mayor que cero y su valor está almacenado en el registro **%cx**. Utiliza un MÁXIMO DE 3 INSTRUCCIONES. Indica claramente las expresiones aritméticas que utilizas en el cálculo de la dirección.

Las expresiones aritméticas para el cálculo son:

$$@k + 2 + i * 22 + 2 + [(i - 1) * 3 + (i - 1)] * 2 = \%edx + 30 * i - 4$$

La sentencia puede codificarse con las siguientes instrucciones:

```
imulw $15, %cx
```

```
movzwl %cx, %ecx
```

```
movw $1, -4(%edx, %ecx, 2)
```


COGNOMS:

[illegible]

NOM:

[illegible]

Problema 4. (2.5 puntos)

Un centro de datos está evaluando la actualización de sus servidores. Actualmente utilizan el Procesador Alfa, pero consideran sustituirlo por el nuevo Procesador Beta. Los datos de ambos procesadores y el IPC (instrucciones por ciclo) de una aplicación crítica de simulación (S) se presentan a continuación

Procesador	Frecuencia (F)	Voltaje (V)	Corriente fuga (I)	Capacidad Equiv. (C)	Cores	IPC
Alfa	3,2 GHz	1,2 V	2,4 A	2,5 nF	16	1,25
Beta	4,0 GHz	1,0 V	1,6 A	2,0 nF	64	2,00

La aplicación S ejecuta 12×10^9 instrucciones dinámicas en un solo core.

a) **Calcula el speed-up** del procesador Beta respecto al Alfa, y los MIPS del procesador Beta usando un solo core.

Texec_Alfa = $12 \times 10^9 / (1,25 \times 3,2 \times 10^9) = 3,0 \text{ s}$
 Texec_Beta = $12 \times 10^9 / (2,00 \times 4,0 \times 10^9) = 1,5 \text{ s}$
 Speedup = $3 \text{ s} / 1,5 \text{ s} = \mathbf{2x}$
 MIPS_Beta = $2,00 \times 4,0 \times 10^9 / 10^6 = \mathbf{8000 \text{ MIPS}}$

Los valores de corriente y capacidad se refieren a un único core. La corriente de cortocircuito es despreciable, por lo que no la tendremos en cuenta. Los cores no activos están apagados y no consumen.

b) **Calcula** la ganancia en energía del Procesador Beta respecto al Alfa al ejecutar la aplicación S en un solo core.

Potencia = Potencia conmutación + potencia fugas = $CV^2F + VI$
 $P_{Alfa} = 2,5 \cdot 1,2^2 \cdot 3,2 + 1,2 \cdot 2,4 = 11,52 + 2,88 = 14,4W$ --- $E_{Alfa} = 14,4 \cdot 3 = 43,2 J$
 $P_{Beta} = 2 \cdot 1,0^2 \cdot 4,0 + 1,0 \cdot 1,6 = 8 + 1,6 = 9,6W$ ----- $E_{Beta} = 9,6 \cdot 1,5 = 14,4 J$
 Ganancia energía = $43,2 / 14,4 = 3x$

Se ha optimizado la aplicación S para que el 96% de su código sea perfectamente paralelizable y la ejecutamos en el procesador Alfa.

c) **Calcula** el speed-up global alcanzado respecto a la ejecución secuencial en el procesador Alfa.

Ley de Amdahl
 $S = 1/(1-f+f/N) = 1/(0,04+0,96/16) = 10x$

El sistema de refrigeración del servidor solo puede disipar un máximo de 300 W. El Procesador Beta soporta escalado dinámico de voltaje y frecuencia (DVFS): por cada 1 GHz que se reduce la frecuencia respecto a su valor nominal (**4 GHz**), el voltaje se reduce automáticamente en 0,1 V. El procesador solo puede operar a frecuencias enteras (1, 2, 3 o 4 GHz).

d) **Calcula** la máxima frecuencia a la que pueden operar los 64 cores simultáneamente sin superar el límite de 300 W.

$P = CV^2F + VI$
 A 4 GHz y 1,0 V (Nominal): 9,6W x64 cores --> 614,4 W
 A 3 GHz y 0,9 V: $2 \times 0,9^2 \times 3,0 + 0,9 \times 1,6 = 6,30$ W x64 cores --> 403,2 W
 A 2 GHz y 0,8 V: $2 \times 0,8^2 \times 2,0 + 0,8 \times 1,6 = 3,84$ W x64 cores --> 245,76 W
 Con 64 cores activos deberá operar a **2 GHz**

(aun así, consigue un speed-up de 2,86x respecto al Alfa, se sugiere como ejercicio que lo calculéis)